

Project Overview:

This assessment focused on implementing layered defensive controls using Windows Defender Firewall, Linux iptables, Snort IDS, and a Pentbox honeypot. The goal was to restrict unauthorized access, monitor suspicious traffic, and collect intelligence on potential attacker behavior.

Tools Used

- Windows Defender Firewall
- Linux iptables
- Snort IDS
- Pentbox Honeypot
- Wget
- Linux Terminal

Skills Demonstrated

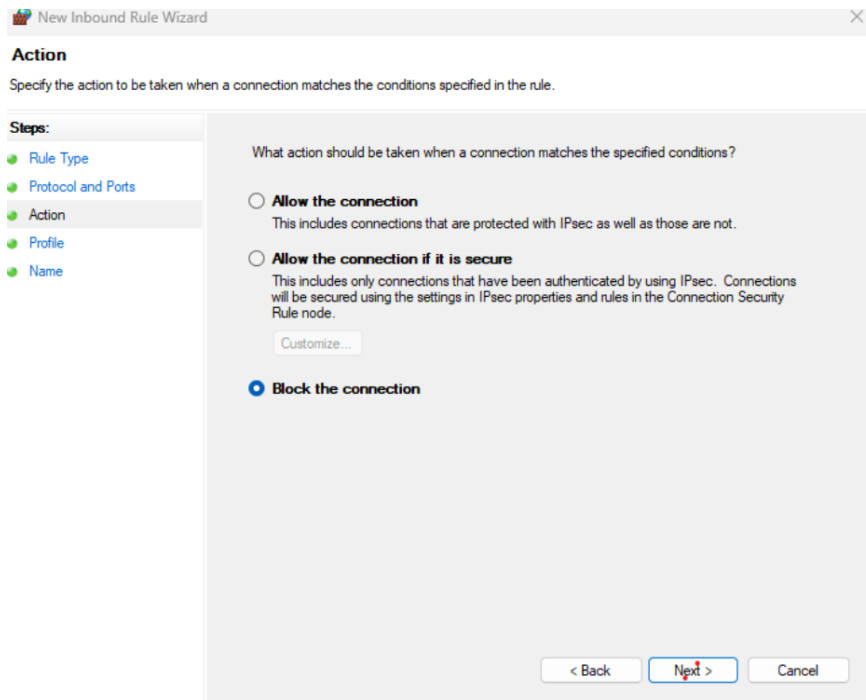
- Firewall Administration
- Access Control
- Intrusion Detection
- Network Monitoring
- Honeypot Deployment
- Security Operations

Summary:

The project demonstrated a defense-in-depth approach by combining host-based firewalls, Linux packet filtering, intrusion detection, and deception technology. Access restrictions were validated through network testing, Snort generated alerts from monitored traffic, and the honeypot illustrated how decoy services can collect intelligence without exposing production systems.

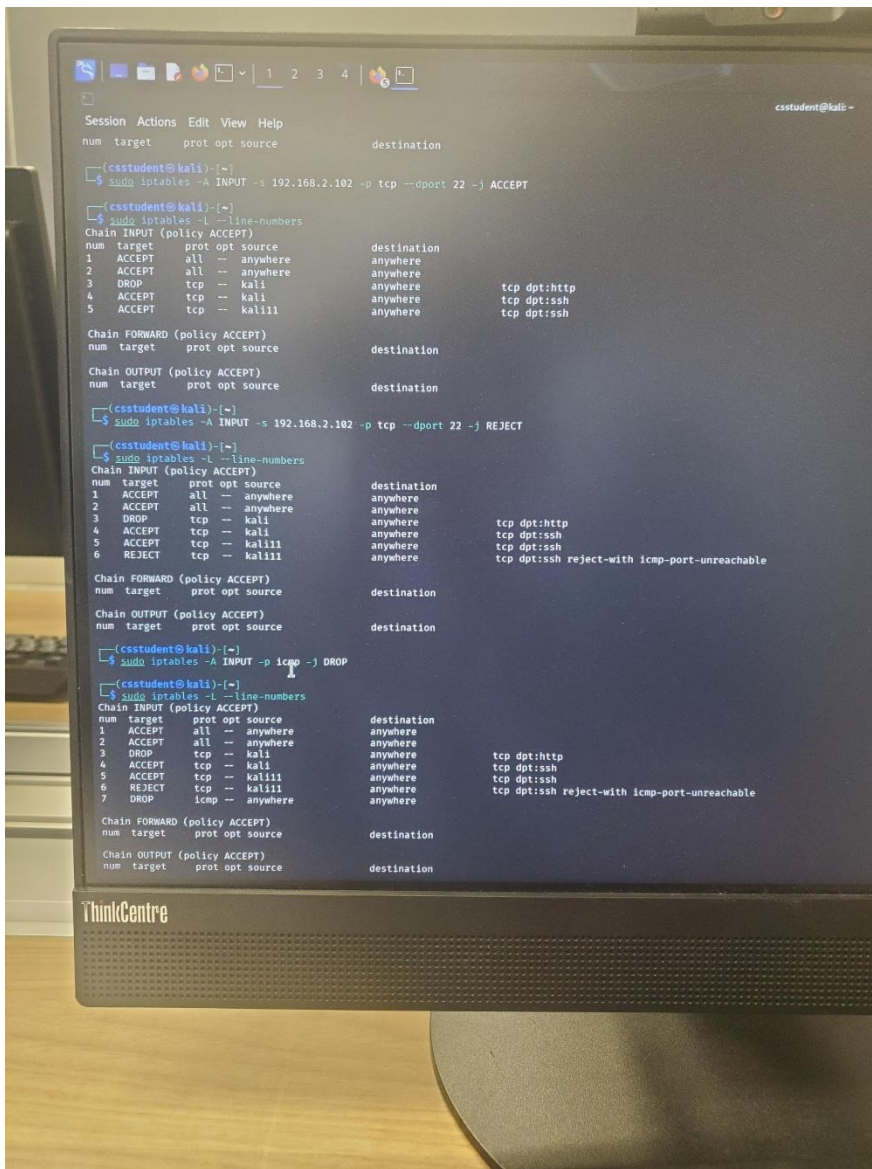
1. Windows Firewall Hardening:

Windows Defender Firewall with Advanced Security was used to create inbound rules that block HTTP/HTTPS traffic to the host. Unauthorized inbound connections were successfully blocked, demonstrating the effectiveness of host-based firewall controls.



2. Linux Access Control with iptables:

Iptables rules were used to drop inbound SSH traffic and selectively allow or deny ICMP traffic depending on the test scenario. The rules enforced granular control over remote administration access and ICMP handling. I would block ssh access to my machine by going to terminal and using this `sudo iptables -A INPUT -p tcp --dport 22 -j DROP` to block all ssh access to my machine. How I would allow ICMP by using this `sudo iptables -A INPUT -p icmp -j ACCEPT` and how I would not allow ICMP is by using this `sudo iptables -A INPUT -p icmp -j DROP`.



3. Access Verification:

Connectivity tests using Wget and related network utilities were performed from another host. Restricted services were inaccessible to unauthorized systems, confirming effective enforcement. Wget shows 192.168.2.104 does not have access.

```
Session Actions Edit View Help
(csstudent@kali)-[~]
$ nmap -sS
Starting Nmap 7.98 ( https://nmap.org ) at 2026-04-02 13:38 -0500
WARNING: No targets were specified, so 0 hosts scanned.
Nmap done: 0 IP addresses (0 hosts up) scanned in 0.05 seconds

(csstudent@kali)-[~]
$ ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.2.104 netmask 255.255.255.0 broadcast 192.168.2.255
    inet6 fe80::9aee:cbff:feb0:3ee5 prefixlen 64 scopeid 0<20<link>
    ether 98:ee:cb:b0:3e:e5 txqueuelen 1000 (Ethernet)
    RX packets 149055 bytes 136149161 (129.8 MiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 78564 bytes 17913874 (17.0 MiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0<10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 330 bytes 52589 (51.3 KiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 330 bytes 52589 (51.3 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

wlan0: flags=4099<UP,BROADCAST,MULTICAST> mtu 1500
    ether 42:e7:60:7a:e6:e9 txqueuelen 1000 (Ethernet)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

(csstudent@kali)-[~]
$ nmap -sS 192.168.2.0/24
Starting Nmap 7.98 ( https://nmap.org ) at 2026-04-02 13:39 -0500
Nmap scan report for gateway (192.168.2.1)
Host is up (0.00017s latency).
Not shown: 997 filtered tcp ports (no-response)
PORT      STATE SERVICE
53/tcp    open  domain
80/tcp    open  http
443/tcp   open  https
MAC Address: EB:EA:6A:09:69:5F (StarTech.com)

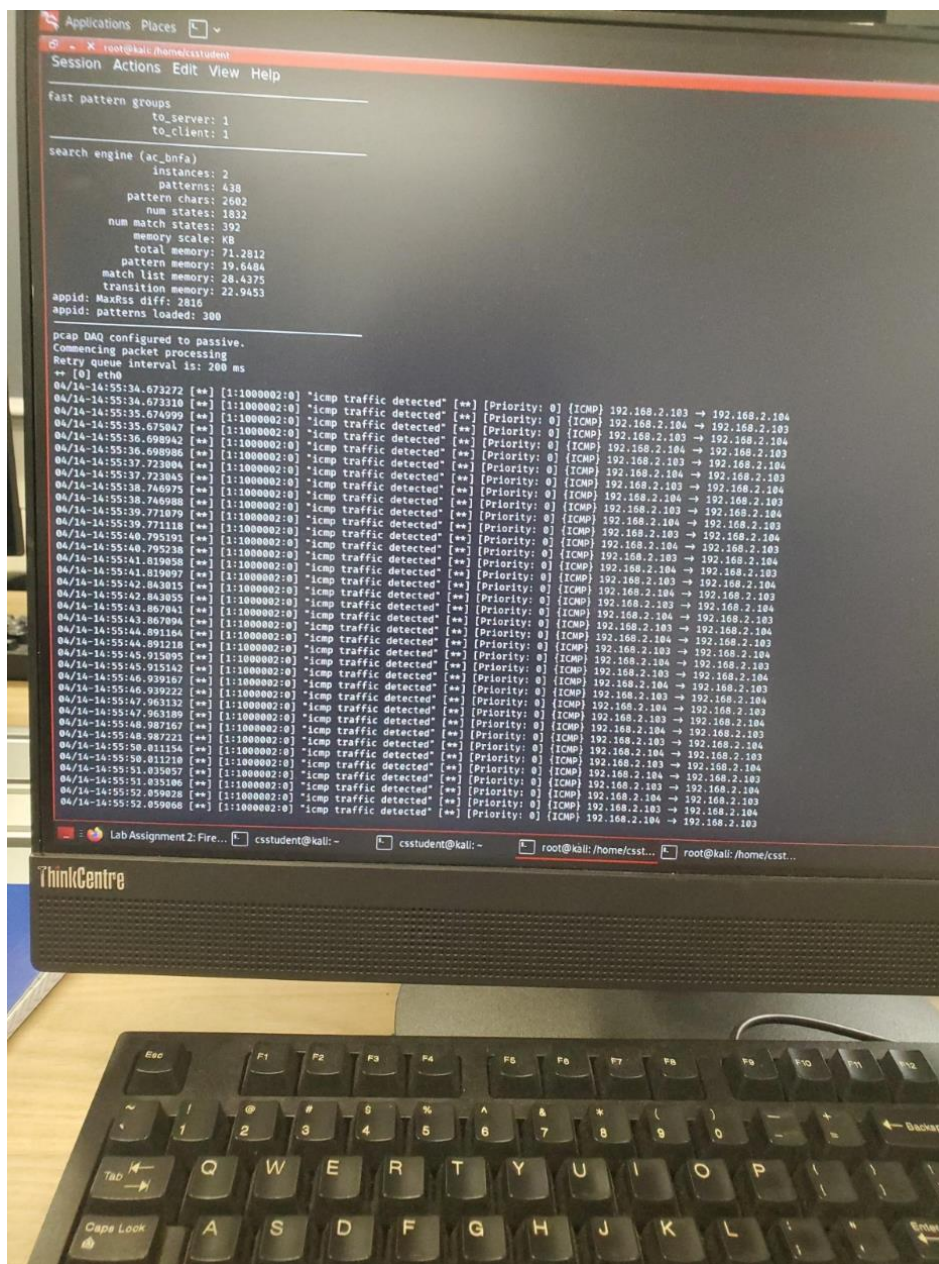
Nmap scan report for kali6.local (192.168.2.104)
Host is up (0.000038s latency).
Not shown: 999 filtered tcp ports (no-response)
PORT      STATE SERVICE
80/tcp    open  http

Nmap done: 256 IP addresses (2 hosts up) scanned in 11.77 seconds

(csstudent@kali)-[~]
```

4. IDS Deployment:

Snort was configured with detection rules to identify specific traffic patterns and raise alerts for monitored events. The IDS generated alerts according to rule conditions, demonstrating visibility into network activity.



5. Honeypot Deployment:

A Pentbox honeypot was configured to simulate vulnerable services and log connection attempts. The honeypot illustrated how deception technologies can gather intelligence on reconnaissance and attack behavior while isolating production assets.

Lesson Learned:

The lab reinforced that no single control is sufficient. Firewalls restrict access, IDS provides visibility, and honeypots supply threat intelligence. Combining these controls improves detection and response capability.

Conclusion:

This assessment provided practical experience with host-based firewalls, Linux packet filtering, intrusion detection, and deception technology. The project strengthened defensive security skills directly relevant to SOC analyst and cybersecurity analyst roles.